

Operators and Expressions (Continued)

Operators in C allow you to manipulate variables and values through various operations. Understanding them is crucial for effective programming.

Arithmetic Operators

Operator	Description	Example
<code>+</code>	Addition	<code>a + b</code>
<code>-</code>	Subtraction	<code>a - b</code>
<code>*</code>	Multiplication	<code>a * b</code>
<code>/</code>	Division	<code>a / b</code>
<code>%</code>	Modulus (remainder)	<code>a % b</code>
<code>++</code>	Increment	<code>a++</code> or <code>++a</code>
<code>--</code>	Decrement	<code>a--</code> or <code>--a</code>

Examples:

```
c
#include <stdio.h>

int main() {
    int a = 10, b = 3;

    printf("Addition: %d + %d = %d\n", a, b, a + b);           // 13
    printf("Subtraction: %d - %d = %d\n", a, b, a - b);       // 7
    printf("Multiplication: %d * %d = %d\n", a, b, a * b);     // 30
    printf("Division: %d / %d = %d\n", a, b, a / b);           // 3 (integer division)
    printf("Modulus: %d %% %d = %d\n", a, b, a % b);           // 1

    int c = a++; // Post-increment: c = 10, a = 11
    printf("Post-increment: c = %d, a = %d\n", c, a);

    int d = ++b; // Pre-increment: d = 4, b = 4
    printf("Pre-increment: d = %d, b = %d\n", d, b);

    return 0;
}
```

Integer vs Floating-point Division

c

```
#include <stdio.h>

int main() {
    int a = 5, b = 2;
    float c = 5.0, d = 2.0;

    printf("Integer division: %d / %d = %d\n", a, b, a / b);           // 2
    printf("Float division: %.1f / %.1f = %.1f\n", c, d, c / d);     // 2.5
    printf("Mixed division: %d / %.1f = %.1f\n", a, d, a / d);       // 2.5

    // Type casting for division
    printf("Cast division: (float)%d / %d = %.1f\n", a, b, (float)a / b); // 2.5

    return 0;
}
```

Relational Operators

Relational operators compare values and return a boolean result (1 for true, 0 for false).

Operator	Description	Example
<code>==</code>	Equal to	<code>a == b</code>
<code>!=</code>	Not equal to	<code>a != b</code>
<code>></code>	Greater than	<code>a > b</code>
<code><</code>	Less than	<code>a < b</code>
<code>>=</code>	Greater than or equal to	<code>a >= b</code>
<code><=</code>	Less than or equal to	<code>a <= b</code>

Example:

c

```
#include <stdio.h>

int main() {
    int a = 10, b = 20;

    printf("%d == %d is %d\n", a, b, a == b); // 0 (false)
    printf("%d != %d is %d\n", a, b, a != b); // 1 (true)
    printf("%d > %d is %d\n", a, b, a > b);    // 0 (false)
    printf("%d < %d is %d\n", a, b, a < b);    // 1 (true)
    printf("%d >= %d is %d\n", a, b, a >= b); // 0 (false)
    printf("%d <= %d is %d\n", a, b, a <= b); // 1 (true)

    return 0;
}
```

Logical Operators

Logical operators combine boolean values.

Operator	Description	Example
&&	Logical AND	a && b
	Logical OR	a b
!	Logical NOT	!a

Example:

C

```
#include <stdio.h>

int main() {
    int a = 1, b = 0;  // In C, 0 is false, non-zero is true

    printf("%d && %d is %d\n", a, b, a && b);    // 0 (false)
    printf("%d || %d is %d\n", a, b, a || b);    // 1 (true)
    printf("!%d is %d\n", a, !a);                // 0 (false)
    printf("!%d is %d\n", b, !b);                // 1 (true)

    // Short-circuit evaluation
    int x = 5, y = 0;
    // Second part not evaluated if first part is false (AND)
    if (y != 0 && x/y > 2) {
        printf("This won't execute\n");
    }

    // Second part not evaluated if first part is true (OR)
    if (x > 0 || x/y > 2) {
        printf("This will execute safely\n");
    }

    return 0;
}
```

Bitwise Operators

Bitwise operators manipulate individual bits in integer values.

Operator	Description	Example
<code>&</code>	Bitwise AND	<code>a & b</code>
<code> </code>	Bitwise OR	<code>a b</code>
<code>^</code>	Bitwise XOR	<code>a ^ b</code>
<code>~</code>	Bitwise NOT	<code>~a</code>
<code><<</code>	Left shift	<code>a << n</code>
<code>>></code>	Right shift	<code>a >> n</code>

Example:

c

```
#include <stdio.h>

int main() {
    unsigned int a = 60; // 00111100 in binary
    unsigned int b = 13; // 00001101 in binary

    printf("a = %u (binary: %08x)\n", a, a);
    printf("b = %u (binary: %08x)\n", b, b);

    printf("a & b = %u\n", a & b); // 00001100 = 12
    printf("a | b = %u\n", a | b); // 00111101 = 61
    printf("a ^ b = %u\n", a ^ b); // 00110001 = 49
    printf("~a = %u\n", ~a); // 11000011... = 4294967235 (32-bit)
    printf("a << 2 = %u\n", a << 2); // 11110000 = 240
    printf("a >> 2 = %u\n", a >> 2); // 00001111 = 15

    return 0;
}
```

Bitwise operators are useful for:

1. Flag manipulation (setting/clearing specific bits)
2. Resource-efficient storage of boolean values
3. Low-level hardware manipulation
4. Optimizing certain mathematical operations

Assignment Operators

Assignment operators assign values to variables, often combining assignment with another operation.

Operator	Description	Equivalent to
=	Simple assignment	-
+=	Add and assign	a = a + b
-=	Subtract and assign	a = a - b
*=	Multiply and assign	a = a * b
/=	Divide and assign	a = a / b
%=	Modulus and assign	a = a % b
&=	Bitwise AND and assign	a = a & b
=	Bitwise OR and assign	a = a b
^=	Bitwise XOR and assign	a = a ^ b
<<=	Left shift and assign	a = a << b
>>=	Right shift and assign	a = a >> b

Example:

```
c
#include <stdio.h>

int main() {
    int a = 10;

    a += 5; // a = a + 5
    printf("After a += 5: a = %d\n", a); // 15

    a -= 3; // a = a - 3
    printf("After a -= 3: a = %d\n", a); // 12

    a *= 2; // a = a * 2
    printf("After a *= 2: a = %d\n", a); // 24

    a /= 4; // a = a / 4
    printf("After a /= 4: a = %d\n", a); // 6

    a %= 4; // a = a % 4
    printf("After a %= 4: a = %d\n", a); // 2

    return 0;
}
```

Conditional (Ternary) Operator

The conditional operator (?:) provides a shorthand way to write simple if-else statements.

Syntax: `condition ? expression_if_true : expression_if_false`

Example:

c

```
#include <stdio.h>

int main() {
    int age = 20;

    // Using if-else
    if (age >= 18) {
        printf("Adult\n");
    } else {
        printf("Minor\n");
    }

    // Using ternary operator
    printf("%s\n", age >= 18 ? "Adult" : "Minor");

    // Assigning values based on condition
    int ticket_price = age < 12 ? 5 : (age < 65 ? 10 : 7);
    printf("Ticket price: $%d\n", ticket_price);

    return 0;
}
```

Special Operators

1. Sizeof Operator

Returns the size of a variable or data type in bytes.

C

```
#include <stdio.h>

int main() {
    int a;
    float b;
    double c;
    char d;

    printf("Size of int: %zu bytes\n", sizeof(a));    // Typically 4
    printf("Size of float: %zu bytes\n", sizeof(b));  // Typically 4
    printf("Size of double: %zu bytes\n", sizeof(c)); // Typically 8
    printf("Size of char: %zu bytes\n", sizeof(d));   // 1
    printf("Size of array: %zu bytes\n", sizeof(int[10])); // Typically 40

    return 0;
}
```

2. Address (&) and Dereference (*) Operators

The address operator (&) returns the memory address of a variable. The dereference operator (*) accesses the value at a memory address.

C

```
#include <stdio.h>

int main() {
    int x = 10;
    int *ptr;

    ptr = &x; // ptr now holds the address of x

    printf("Value of x: %d\n", x);          // 10
    printf("Address of x: %p\n", (void*)&x); // Memory address (e.g., 0x7fffedb0dc4c)
    printf("Value of ptr: %p\n", (void*)ptr); // Same address as &x
    printf("Value *ptr points to: %d\n", *ptr); // 10

    *ptr = 20; // Change the value at the address ptr points to
    printf("After *ptr = 20, x = %d\n", x);  // 20

    return 0;
}
```

3. Structure Member Operators (.) and (->)

Access members of structures and unions.

c

```
#include <stdio.h>

struct Person {
    char name[50];
    int age;
};

int main() {
    struct Person person1 = {"John", 25};
    struct Person *ptr = &person1;

    // Dot operator for direct access
    printf("Name: %s, Age: %d\n", person1.name, person1.age);

    // Arrow operator for pointer access
    printf("Name: %s, Age: %d\n", ptr->name, ptr->age);

    // Equivalent way using dereference and dot
    printf("Name: %s, Age: %d\n", (*ptr).name, (*ptr).age);

    return 0;
}
```

Operator Precedence and Associativity

Operator precedence determines the order in which operations are performed. Higher precedence operators are evaluated first.

Here's a simplified precedence table (from highest to lowest):

1. Postfix operators: `()`, `[]`, `->`, `.`, `++` (postfix), `--` (postfix)
2. Prefix operators: `++` (prefix), `--` (prefix), `+` (unary), `-` (unary), `!`, `~`, `(type)`, `*`, `&`, `sizeof`
3. Multiplicative: `*`, `/`, `%`
4. Additive: `+`, `-`
5. Shift: `<<`, `>>`
6. Relational: `<`, `<=`, `>`, `>=`
7. Equality: `==`, `!=`
8. Bitwise AND: `&`
9. Bitwise XOR: `^`

- 10. Bitwise OR: `|`
- 11. Logical AND: `&&`
- 12. Logical OR: `||`
- 13. Conditional: `?:`
- 14. Assignment: `=`, `+=`, `-=`, etc.
- 15. Comma: `,`

Example demonstrating precedence:

c

```
#include <stdio.h>
```

```
int main() {
```

```
    int a = 5, b = 3, c = 8, result;
```

```
    // Without parentheses (following precedence rules)
```

```
    result = a + b * c;
```

```
    printf("a + b * c = %d\n", result); // 5 + (3 * 8) = 29
```

```
    // With parentheses (changing the order)
```

```
    result = (a + b) * c;
```

```
    printf("(a + b) * c = %d\n", result); // (5 + 3) * 8 = 64
```

```
    // Complex expression
```

```
    result = a * b + c / 2 - 1;
```

```
    printf("a * b + c / 2 - 1 = %d\n", result); // (5 * 3) + (8 / 2) - 1 = 15 + 4 - 1 = 18
```

```
    return 0;
```

```
}
```

Common Expressions and Their Uses

1. Arithmetic Expressions

C

```
#include <stdio.h>
#include <math.h> // For mathematical functions

int main() {
    // Area of a circle
    double radius = 5.0;
    double area = M_PI * radius * radius; // or M_PI * pow(radius, 2)
    printf("Area of circle: %.2f\n", area); // 78.54

    // Temperature conversion
    double celsius = 25.0;
    double fahrenheit = (celsius * 9.0/5.0) + 32.0;
    printf("%.1f°C = %.1f°F\n", celsius, fahrenheit); // 25.0°C = 77.0°F

    // Average of values
    int values[] = {12, 45, 67, 23, 9};
    int sum = 0;
    for (int i = 0; i < 5; i++) {
        sum += values[i];
    }
    double average = (double)sum / 5;
    printf("Average: %.2f\n", average); // 31.20

    return 0;
}
```

2. Logical Expressions

c

```
#include <stdio.h>

int main() {
    int age = 25;
    float gpa = 3.5;

    // Checking multiple conditions
    if (age >= 18 && age <= 30) {
        printf("Young adult\n");
    }

    // Checking inclusive ranges
    if (gpa >= 3.0 && gpa <= 4.0) {
        printf("Good academic standing\n");
    }

    // Checking exclusive conditions
    char grade = 'B';
    if (grade == 'A' || grade == 'B') {
        printf("Above average performance\n");
    }

    // Negating conditions
    int is_weekend = 0; // 0 = false, 1 = true
    if (!is_weekend) {
        printf("It's a weekday\n");
    }

    return 0;
}
```

3. Bitwise Manipulation Expressions

c

```
#include <stdio.h>

int main() {
    // Setting a bit
    unsigned char flags = 0; // 00000000
    flags |= (1 << 2);        // Set bit 2: 00000100
    printf("After setting bit 2: %d\n", flags); // 4

    // Clearing a bit
    flags &= ~(1 << 2);        // Clear bit 2: 00000000
    printf("After clearing bit 2: %d\n", flags); // 0

    // Toggling a bit
    flags ^= (1 << 3);        // Toggle bit 3: 00001000
    printf("After toggling bit 3: %d\n", flags); // 8

    // Checking if a bit is set
    flags = 12; // 00001100
    if (flags & (1 << 2)) {
        printf("Bit 2 is set\n"); // Will print
    }

    if (flags & (1 << 0)) {
        printf("Bit 0 is set\n"); // Won't print
    }

    return 0;
}
```

Example: Calculator Program Using Various Operators


```

#include <stdio.h>
#include <math.h>

int main() {
    double num1, num2, result;
    char operator;

    printf("Enter a simple expression (e.g., 5 + 3): ");
    scanf("%lf %c %lf", &num1, &operator, &num2);

    switch (operator) {
        case '+':
            result = num1 + num2;
            printf("%.2f + %.2f = %.2f\n", num1, num2, result);
            break;
        case '-':
            result = num1 - num2;
            printf("%.2f - %.2f = %.2f\n", num1, num2, result);
            break;
        case '*':
            result = num1 * num2;
            printf("%.2f * %.2f = %.2f\n", num1, num2, result);
            break;
        case '/':
            if (num2 != 0) {
                result = num1 / num2;
                printf("%.2f / %.2f = %.2f\n", num1, num2, result);
            } else {
                printf("Error: Division by zero\n");
            }
            break;
        case '%':
            if (num2 != 0) {
                result = fmod(num1, num2); // Use fmod for floating-point modulus
                printf("%.2f %% %.2f = %.2f\n", num1, num2, result);
            } else {
                printf("Error: Division by zero\n");
            }
            break;
        case '^':
            result = pow(num1, num2);
            printf("%.2f ^ %.2f = %.2f\n", num1, num2, result);
            break;
        default:
            printf("Error: Unsupported operator\n");
    }
}

```

```
    return 0;  
}
```

Programming using Variables, Data Types, Operators, and I/O Functions

Let's put all the concepts learned so far into practice with some comprehensive examples:

Example 1: Temperature Conversion Program

This program converts temperatures between Celsius and Fahrenheit:

C

```
#include <stdio.h>

int main() {
    float celsius, fahrenheit;
    char choice;

    printf("Temperature Conversion Program\n");
    printf("-----\n");
    printf("Enter 'C' to convert Fahrenheit to Celsius\n");
    printf("Enter 'F' to convert Celsius to Fahrenheit\n");
    printf("Your choice: ");
    scanf(" %c", &choice);

    switch (choice) {
        case 'C':
        case 'c':
            printf("Enter temperature in Fahrenheit: ");
            scanf("%f", &fahrenheit);
            celsius = (fahrenheit - 32) * 5 / 9;
            printf("%.2f°F = %.2f°C\n", fahrenheit, celsius);
            break;

        case 'F':
        case 'f':
            printf("Enter temperature in Celsius: ");
            scanf("%f", &celsius);
            fahrenheit = (celsius * 9 / 5) + 32;
            printf("%.2f°C = %.2f°F\n", celsius, fahrenheit);
            break;

        default:
            printf("Invalid choice!\n");
    }

    return 0;
}
```

Example 2: Simple Banking System

This program demonstrates a simple bank account system:


```
#include <stdio.h>

int main() {
    double balance = 1000.0; // Initial balance
    int choice;
    double amount;

    printf("Simple Banking System\n");
    printf("-----\n");
    printf("Current Balance: $%.2f\n\n", balance);

    do {
        printf("1. Deposit\n");
        printf("2. Withdraw\n");
        printf("3. Check Balance\n");
        printf("4. Exit\n");
        printf("Enter your choice (1-4): ");
        scanf("%d", &choice);

        switch (choice) {
            case 1: // Deposit
                printf("Enter deposit amount: $");
                scanf("%lf", &amount);

                if (amount > 0) {
                    balance += amount;
                    printf("$%.2f deposited successfully.\n", amount);
                } else {
                    printf("Invalid amount. Please enter a positive value.\n");
                }
                break;

            case 2: // Withdraw
                printf("Enter withdrawal amount: $");
                scanf("%lf", &amount);

                if (amount > 0) {
                    if (amount <= balance) {
                        balance -= amount;
                        printf("$%.2f withdrawn successfully.\n", amount);
                    } else {
                        printf("Insufficient funds. Your balance is $%.2f\n", balance);
                    }
                } else {
                    printf("Invalid amount. Please enter a positive value.\n");
                }
            }
        }
    }
```

```
        break;

    case 3: // Check Balance
        printf("Current Balance: $%.2f\n", balance);
        break;

    case 4: // Exit
        printf("Thank you for using our banking system!\n");
        break;

    default:
        printf("Invalid choice. Please select 1-4.\n");
}

printf("\n");
} while (choice != 4);

return 0;
}
```

Example 3: Student Grade Calculator

This program calculates and displays student grades:


```
#include <stdio.h>

int main() {
    const int MAX_SUBJECTS = 5;
    int marks[MAX_SUBJECTS];
    int total = 0;
    float average;
    char grade;

    printf("Student Grade Calculator\n");
    printf("-----\n");

    // Input marks for each subject
    for (int i = 0; i < MAX_SUBJECTS; i++) {
        do {
            printf("Enter marks for Subject %d (0-100): ", i + 1);
            scanf("%d", &marks[i]);

            if (marks[i] < 0 || marks[i] > 100) {
                printf("Invalid marks. Please enter between 0 and 100.\n");
            }
        } while (marks[i] < 0 || marks[i] > 100);

        total += marks[i];
    }

    // Calculate average
    average = (float)total / MAX_SUBJECTS;

    // Determine grade
    if (average >= 90) {
        grade = 'A';
    } else if (average >= 80) {
        grade = 'B';
    } else if (average >= 70) {
        grade = 'C';
    } else if (average >= 60) {
        grade = 'D';
    } else {
        grade = 'F';
    }

    // Display results
    printf("\nResults:\n");
    printf("-----\n");
```

```
for (int i = 0; i < MAX_SUBJECTS; i++) {  
    printf("Subject %d: %d\n", i + 1, marks[i]);  
}  
  
printf("\nTotal Marks: %d out of %d\n", total, MAX_SUBJECTS * 100);  
printf("Average: %.2f%%\n", average);  
printf("Grade: %c\n", grade);  
  
if (grade == 'F') {  
    printf("Status: Failed\n");  
} else {  
    printf("Status: Passed\n");  
}  
  
return 0;  
}
```

Example 4: Simple Calculator with Memory Functions

This calculator program demonstrates the use of memory variables and mathematical operations:


```

#include <stdio.h>
#include <math.h>

int main() {
    double num1, num2, result = 0;
    double memory = 0; // Memory variable
    char operator;
    int choice;

    printf("Advanced Calculator with Memory\n");
    printf("-----\n");

    do {
        printf("\nCurrent Result: %.2f\n", result);
        printf("Memory Value: %.2f\n\n", memory);

        printf("1. Perform calculation\n");
        printf("2. Store to memory\n");
        printf("3. Recall from memory\n");
        printf("4. Clear memory\n");
        printf("5. Clear result\n");
        printf("6. Exit\n");
        printf("Enter your choice (1-6): ");
        scanf("%d", &choice);

        switch (choice) {
            case 1: // Calculation
                printf("Enter operation (e.g., 5 + 3, or 'r' for result): ");
                scanf(" %c", &operator);

                if (operator == 'r' || operator == 'R') {
                    num1 = result;
                } else {
                    scanf("%lf", &num1);
                }

                scanf(" %c", &operator);
                scanf("%lf", &num2);

                switch (operator) {
                    case '+':
                        result = num1 + num2;
                        break;
                    case '-':
                        result = num1 - num2;
                        break;

```

```

        case '*':
            result = num1 * num2;
            break;
        case '/':
            if (num2 != 0) {
                result = num1 / num2;
            } else {
                printf("Error: Division by zero\n");
                continue;
            }
            break;
        case '^':
            result = pow(num1, num2);
            break;
        case '%':
            if (num2 != 0) {
                result = fmod(num1, num2);
            } else {
                printf("Error: Division by zero\n");
                continue;
            }
            break;
        default:
            printf("Error: Unsupported operator\n");
            continue;
    }

    printf("%.2f %c %.2f = %.2f\n", num1, operator, num2, result);
    break;

case 2: // Store to memory
    memory = result;
    printf("Value %.2f stored to memory\n", result);
    break;

case 3: // Recall from memory
    result = memory;
    printf("Value %.2f recalled from memory\n", memory);
    break;

case 4: // Clear memory
    memory = 0;
    printf("Memory cleared\n");
    break;

case 5: // Clear result
    result = 0;

```

```

        printf("Result cleared\n");
        break;

    case 6: // Exit
        printf("Exiting calculator...\n");
        break;

    default:
        printf("Invalid choice. Please select 1-6.\n");
    }
} while (choice != 6);

return 0;
}

```

Control Structures - Selection Statements and Looping

Control structures allow you to control the flow of execution in a program. They are divided into two main categories: selection (decision-making) and iteration (looping).

Selection Statements

1. `if` Statement

The `if` statement executes a block of code if a specified condition is true.

```

c
if (condition) {
    // Code to execute if condition is true
}

```

Example:

c

```
#include <stdio.h>

int main() {
    int age;

    printf("Enter your age: ");
    scanf("%d", &age);

    if (age >= 18) {
        printf("You are eligible to vote.\n");
    }

    return 0;
}
```

2. **if-else** Statement

The **if-else** statement executes one block of code if a condition is true and another block if it is false.

c

```
if (condition) {
    // Code to execute if condition is true
} else {
    // Code to execute if condition is false
}
```

Example:

c

```
#include <stdio.h>

int main() {
    int number;

    printf("Enter a number: ");
    scanf("%d", &number);

    if (number % 2 == 0) {
        printf("%d is an even number.\n", number);
    } else {
        printf("%d is an odd number.\n", number);
    }

    return 0;
}
```

3. **if-else if-else** Ladder

The **if-else if-else** ladder checks multiple conditions in sequence.

c

```
if (condition1) {
    // Code to execute if condition1 is true
} else if (condition2) {
    // Code to execute if condition1 is false and condition2 is true
} else if (condition3) {
    // Code to execute if condition1 and condition2 are false and condition3 is true
} else {
    // Code to execute if all conditions are false
}
```

Example:

c

```
#include <stdio.h>

int main() {
    int score;

    printf("Enter your exam score (0-100): ");
    scanf("%d", &score);

    if (score >= 90) {
        printf("Grade: A\n");
    } else if (score >= 80) {
        printf("Grade: B\n");
    } else if (score >= 70) {
        printf("Grade: C\n");
    } else if (score >= 60) {
        printf("Grade: D\n");
    }
}
```